

Project Report

Design and Implementation of a Turkish-Inspired Programming Language

Course: Programming Languages

Project Name: C-TR-L

Due Date: May 31, 2026

Group Members

Yağız Cengiz - 230315076

Kayra Hökenek - 230315048

Mehmet Hamza Öğretmiş - 244101073

Khazar Abdullayev - 244101083

Introduction

This project encompasses the design and implementation of **C-TR-L**, a custom programming language frontend. Built directly upon the theoretical foundations of Chapter 4 in Robert W. Sebesta's *Concepts of Programming Languages*, this study bridges formal language theory with practical compiler engineering by delivering a fully operational lexical analyzer (`lexer.c`) and a recursive-descent syntax analyzer (`parser.c`).

C-TR-L adopts a C-style imperative structure while replacing traditional English tokens with intuitive, Turkish-inspired keywords (such as `tam_sayı`, `eğer`, and `için`). A key technical achievement of our implementation is the custom UTF-8 character classification matrix, which seamlessly integrates multi-byte Turkish localized graphemes (ç, ğ, ı, ö, ş, ü) into user-defined identifiers without corrupting the lexical state machine.

Mechanically, the frontend utilizes a deterministic scanner for token assembly and a top-down recursive-descent parsing strategy to enforce strict syntactic constraints. By successfully resolving structural challenges—such as inherent operator precedence and nested conditionals—this report demonstrates how formal grammatical rules are systematically transformed into a robust, diagnostic verification tool.

Base Textbook Code Used

The architectural blueprint of the **C-TR-L** compiler frontend is derived directly from the foundational source code examples presented in Robert W. Sebesta's *Concepts of Programming Languages* (10th Edition), Chapter 4: Lexical and Syntax Analysis. Specifically, the implementation leverages the state-driven lexical analyzer skeleton spanning pages 172–176 and the top-down recursive-descent parsing framework illustrated from page 182 onwards. To clearly delineate our development pipeline as required by the project specifications, the relationship between the original textbook foundation and our final implementation is categorized into preserved, modified, and newly designed components below:

1. What Was Kept (Preserved Core Foundations)

To maintain structural consistency with Sebesta's architectural model, the global state synchronization mechanism was preserved in its entirety. This includes the reliance on the global character stream pointer (`in_fp`), the active character classification variable (`charClass`), the lookahead token tracker (`nextToken`), and the primary character buffer (`lexeme`).

The foundational subroutines for sequential text streaming were maintained without altering their algorithmic design:

- `getChar()` remains the fundamental entry point for capturing single characters from the input stream.
- `addChar()` is utilized to safely append characters into the lexeme array while preventing buffer overflows.
- `getNonBlank()` continues to serve as the lookahead pre-processor that truncates non-semantic white spaces.

Additionally, the core structural approach to mathematical expression processing within the parser—specifically the cascading left-associative loops inside `expr()`, `term()`, and `factor()`—was retained to guarantee that multiplication, division, addition, and subtraction strictly follow textbook precedence paradigms.

2. What Was Modified (Adapted Architectural Components)

Several core textbook functions were extensively refactored to adapt the scanner and parser from a simple, theoretical English setup into a robust environment compatible with the **C-TR-L** specification:

- **Character Classification Mechanism:** The primitive character routing logic within `getChar()` was overhauled. It was modified to route incoming bytes through a custom `isValidBNFChar()` classification filter. This modification ensures that multi-byte UTF-8 sequences representing Turkish extended graphemes (`uc >= 128`) are correctly classified under the LETTER state machine path rather than being rejected as foreign or illegal characters.
- **Relational Suffix-Matching in `lookup()`:** The single-character evaluation layout of the original `lookup()` function was heavily modified. To support complex relational operations without modifying the main parsing loop, a **1-character lookahead pattern** was implemented. When characters like `=`, `!`, `<`, or `>` are encountered, `lookup()` is modified to dynamically peek at the immediate next byte, using `ungetc()` to safely step back if a compound token (like `==`, `!=`, `<=`, or `>=`) is not completed.

3. What Was Added (Custom Language Extensions)

To fulfill the requirements of a complete, standalone language frontend capable of processing conditional loops and declaring localized variables, the following structural modules were designed and implemented completely from scratch:

- **isKeyword() Subroutine:** A dedicated lexical filter was added to perform string-matching operations via `strcmp()` against our localized language dictionary, cleanly separating reserve commands (`tam_sayı`, `eğer`, `iken`, `için`) from standard user-defined identifiers (IDENT).
- **Advanced Non-Terminal Parsing Subroutines:** Brand-new structural routing routines were engineered within `parser.c` to support the expanded custom CFG rules. These include `program()`, `stmt()`, `declaration()`, `assign()`, and `loop_stmt()`.
- **Nested Conditional Branching (`optional()`):** A highly specialized, recursive subroutine named `optional()` was added to resolve the classic dangling-else problem, allowing the parser to dynamically process alternating `ise` (else-if) cascades and terminate cleanly at a `değilse` (else) terminal or an empty epsilon (`$\epsilon$`) production.
- **Symbol Table Tracking Subsystem:** A centralized repository matrix (`symbolTable`) and a state-checking insertion function (`addSymbol()`) were added to the pipeline. This allows the compiler frontend to track, log, and display user-defined variables upon successful syntax verification.

Overview of the Designed Language

C-TR-L is designed as a strongly typed, imperative, block-structured programming language. Syntactically, it inherits the structural clarity and operational paradigms of traditional C-style languages, but it completely refactors the lexical layer by replacing English reserved words with localized, Turkish-inspired keywords. The primary design goal of **C-TR-L** is to minimize the cognitive overhead and linguistic barriers faced by native Turkish

speakers when interacting with formal language semantics, while maintaining a mathematically rigorous execution environment.

The foundational design of the language is subdivided into four major architectural specifications:

1. Language Paradigm and Syntax Architecture

C-TR-L follows a strict imperative programming paradigm, where execution flows sequentially through explicit statements that modify the internal state of the program. The language enforces block-structured layout conventions using standard delimiters:

- **Parentheses ()** are utilized to rigidly isolate logical expressions within conditional loops and selection statements.
- **Semicolons ;** act as mandatory statement terminators, defining structural boundaries and signaling the parser to clear the temporary lexeme buffer.
- **Variable Scoping and Layout** mimic C-style rules, requiring identifiers to be explicitly declared before they can absorb values or participate in arithmetic computations.

2. Data Typology and Memory Reservation

The type subsystem of **C-TR-L** is deliberately minimal yet internally consistent, focusing entirely on primitive integer representation via the **tam_sayı** keyword. When the compiler frontend encounters a **tam_sayı** instruction, it triggers a memory-reservation sequence. Rather than dynamically resolving types at runtime, **C-TR-L** utilizes static type binding. Every declared variable is immediately checked for namespace collisions and logged into a centralized lookup matrix (the Symbol Table). This guarantees that user-defined variables are strictly bound to integer states, and any illegal attempt to assign foreign literals or malformed symbols is immediately intercepted at the compilation threshold.

3. Control Flow Subsystems (Selection and Iteration)

To support non-linear execution paths and complex logical routing, **C-TR-L** integrates advanced branching and iterative control sub-structures:

- **Conditional Selection (eğer, ise, değilse):** Dynamic branching is initiated using the **eğer** keyword, which evaluates a relational condition enclosed in parentheses. To support complex multi-way branching without structural ambiguity, the language introduces the **ise** clause (equivalent to *else if*) and a final fallback **değilse** block (equivalent to *else*). Syntactically, this design resolves the classic "dangling-else" dilemma by enforcing an explicit recursive grammar route where each alternative branch is bound to its parent conditional structure.
- **Iterative Loops (iken, için):** The language provides two distinct looping mechanisms. The **iken** loop acts as a conditional pre-tested iteration control structure (equivalent to *while*), repeating its body statement as

long as the boundary expression evaluates to true. The **için** loop represents a highly structured, counter-controlled iteration framework (equivalent to *for*). It encapsulates three distinct operations within a single control block: index initialization (`begin_index`), loop termination evaluation (`condition`), and a step modifier loop operation (`operation`).

4. Arithmetic Expression and Relational Subsystems

Computation within **C-TR-L** is handled by a balanced expression evaluation grid designed to implicitly preserve mathematical operator precedence:

- **Arithmetic Expressions:** The language supports binary operators for addition (+), subtraction (-), multiplication (*), and division (/), as well as unary shorthand constructs (++, --) for manual index modification inside loop bodies. The language grammar ensures that high-precedence operators (*, /) are resolved at a lower syntactic layer (factors and terms) before passing execution to low-precedence additive operations (+, -).
- **Relational Assertions:** Logical evaluations rely on a robust subsystem of single and multi-character relational tokens, including equality checking (==), inequality checking (!=), and strict boundary limits (<, >, <=, >=). These multi-byte operator alignments are handled seamlessly by the scanner's lookahead subsystem, allowing the parser to evaluate comparative conditional boundaries accurately.

Reserved Keywords and Tokens

Reserved Keyword	Token Internal Code	Functional Description
tam_sayı	INT_TYPE (27)	Primitive Integer Variable Declaration
eğer	IF (28)	Conditional Clause Block Initialization
ise	ISE (38)	Intermediate Branch Alternative (Else-If)
değilse	ELSE (29)	Default Fallback Condition Block
iken	WHILE (30)	Conditional Pre-tested Loop Structure
için	FOR (39)	Counter-controlled Loop Structure
yazdır	PRINT (31)	Standard Output Stream Command

Operators, Separators, and Generic Tokens

In addition to the reserved keywords, **C-TR-L** recognizes a comprehensive set of mathematical operators, relational boundaries, and structural separators. The lexer also dynamically categorizes user-defined names and raw numbers into generic token classes.

Generic Token Classes:

- **IDENT (11)**: Dynamically assigned to user-defined variables and identifiers (e.g., x, toplam).
- **INT_LIT (10)**: Assigned to raw numerical integer values (e.g., 7, 10).
- **EOF (-1)**: End-of-File marker signaling the parser to successfully terminate execution.

Operators and Structural Tokens Table:

Symbol / Operator	Token Internal Code	Functional Description
=	ASSIGN_OP (20)	Variable Assignment Operator
+, -, *, /	ADD_OP (21), SUB_OP (22), MULT_OP (23), DIV_OP (24)	Standard Binary Arithmetic Operators
++, --	PLUSPLUS (43), MINUSMINUS (44)	Unary Increment and Decrement Operators
==, !=	EQ_OP (32), NEQ_OP (40)	Relational Equality and Inequality Operators
<, >, <=, >=	LT_OP (33), GT_OP (34), LE_OP (41), GE_OP (42)	Relational Boundary Operators
(,)	LEFT_PAREN (25), RIGHT_PAREN (26)	Expression and Condition Delimiters
{, }	LEFT_BRACE (35), RIGHT_BRACE (36)	Block Scope Delimiters
;	SEMICOLON (37)	Statement Terminator

BNF / EBNF Grammar

The syntactic structure of **C-TR-L** is formally defined using Extended Backus-Naur Form (EBNF). In this notation, terminal symbols and reserved keywords are enclosed in quotation marks or written directly (e.g., ";", tam_sayı), while non-terminal abstractions are enclosed in angle brackets (e.g., <stmt>). To construct a concise and unambiguous grammar tree, standard EBNF extensions are utilized: curly braces { } indicate repetitions (zero or more times), and square brackets [] represent optional structural constructs.

```
<program> → <stmt_list> EOF
<stmt> → <declaration> | <assign> | <if_stmt> | <loop_stmt>
<declaration> → tam_sayı <id> ";"
<assign> → <id> "=" <expr> ";"
<if_stmt> → eğer "(" <condition> ")" <statement> [ <optional> ]
<loop_stmt> → iken "(" <condition> ")" <statement>
               | için "(" <begin_index> ";" <condition> ";" <operation> ")"
<statement>
<condition> → <expr> <rel_op> <expr>
<rel_op> → "==" | "!=" | "<" | ">" | "<=" | ">="
<expr> → <term> { ("+" | "-") <term> }
<term> → <factor> { ("*" | "/" ) <factor> }
<factor> → "(" <expr> ")" | <id> | <number>
```

Key Grammatical Features:

- **Inherent Operator Precedence:** Mathematical order of operations is strictly enforced without external parsing logic by stratifying expressions recursively. Base values and parenthetical groups are evaluated first at the <factor> level, followed by high-precedence multiplication and division at the <term> level, and finally low-precedence addition and subtraction at the <expr> level.
- **Dangling-Else Resolution:** The conditional <if_stmt> grammar seamlessly resolves structural ambiguity by utilizing the [<optional>] non-terminal. This ensures that any subsequent ise (else-if) or değilse (else) blocks are explicitly bound to the most recently opened eğer condition.

Lexical Analyzer Design

The lexical analyzer (lexer.c) acts as the foundational frontend component of the **C-TR-L** compiler pipeline. Its primary objective is to read the raw source code character-by-character from the input file pointer (in_fp), filter out non-semantic elements such as white spaces, and group legal character sequences into atomic linguistic units called **lexemes**. Each recognized lexeme is subsequently mapped to an internal integer representation known as a **token code**, which is consumed by the syntax analyzer.

The architecture of the lexical analyzer relies on a deterministic state machine driven by three discrete character classes: LETTER (0), DIGIT (1), and UNKNOWN (99). The operational workflow is split into four highly integrated subroutines:

1. Character Classification Architecture

The `getChar()` function serves as the system's entry point for character acquisition. Unlike standard ASCII scanners, the **C-TR-L** lexer features a custom-designed classification utility called `isValidBNFChar()`. This function ensures full compatibility with Turkish-specific lowercase and uppercase characters (ç, ğ, ı, ö, ş, ü, Ç, Ğ, İ, Ö, Ş, Ü).

Since modern source files are typically encoded in UTF-8, these localized characters span across multiple bytes beyond the standard 7-bit ASCII boundaries. To prevent the compiler from misclassifying these multi-byte structures as illegal or separate symbols, the routine treats any byte value greater than or equal to 128 (`uc >= 128`) as a valid constituent of the LETTER class. Consequently, localized letters are seamlessly aggregated into variable names and reserved language keywords without corrupting the lexical state.

2. Lexeme Assembly Engine

The main driver function, `lex()`, begins execution by calling `getNonBlank()`, which repeatedly invokes `getChar()` to skip spaces, tabs, and newline characters until a semantic character is encountered. Once a non-blank character is isolated, the lexer transitions into its assembly loop based on the active `charClass`:

- **LETTER State:** If the initial character is a letter or an underscore, the scanner enters a loop that continuously appends characters to the lexeme buffer array via `addChar()` as long as the incoming characters remain classified as LETTER or DIGIT.
- **DIGIT State:** If the initial character is a numerical digit, the scanner captures sequential digits to assemble an integer literal (`INT_LIT`), discarding any non-digit boundaries.

3. Keyword vs. Identifier Disambiguation

Once a complete word lexeme is assembled inside the LETTER loop, it undergoes an extraction filter via the `isKeyword()` function. Because keywords and user-defined variables share identical lexical patterns (both start with a letter and contain alphanumeric streams), the lexer performs a string-matching operation using `strcmp()` against the language's reserved dictionary.

If the lexeme matches a reserved string (such as `tam_sayı`, `eğer`, `iken`, or `için`), the token variable `nextToken` is bound to its unique keyword token code (e.g., `INT_TYPE`, `IF`, `WHILE`, `FOR`). If the dictionary search yields no match, the lexeme is safely classified as a standard user-defined variable name, assigning it the generic `IDENT` token code.

4. Lookahead and Suffix-Matching for Operators

When the character class is determined to be UNKNOWN, the stream is directed to the `lookup()` subroutine to identify mathematical, structural, or relational operators. To support compound relational expressions without introducing a complex backtracking engine, the function utilizes a **1-character lookahead (peeking) strategy**.

For instance, when encountering an assignment operator `=`, a inequality marker `!`, or relational boundaries like `<` and `>`, the scanner instantly fetches the immediate next character using `getChar()`. If the subsequent character

completes a valid double-character operator token (such as `==`, `!=`, `<=`, or `>=`), both characters are committed to the lexeme buffer, and the respective compound token code (e.g., `EQ_OP`, `NEQ_OP`) is returned. If the lookahead character does not form a compound operator, it is pushed back into the input stream pointer using the standard `ungetc()` function, ensuring that only the single-character operator is evaluated without altering the subsequent execution path.

Syntax Analyzer Design

The syntax analyzer (`parser.c`) serves as the structural enforcement engine of the **C-TR-L** compiler frontend. Its primary responsibility is to take the stream of tokens produced by the lexical analyzer and verify whether they adhere to the syntactic rules defined by the language's Context-Free Grammar (CFG). To achieve this, the parser implements a deterministic, top-down parsing strategy known as **Recursive-Descent Parsing**, where every non-terminal symbol in the BNF grammar corresponds directly to a specific C function subroutine.

The architecture and operational execution of the syntax analyzer are governed by the following core programmatic mechanisms:

1. Parser-Lexer Co-operation and Token Consumption

The integration between the lexer and the parser is established through a shared global state configuration involving the `nextToken` variable and the lexeme character buffer. Before the parsing tree is initiated, the system undergoes a priming phase inside the `main()` function, where `getChar()` and `lex()` are invoked once to load the very first token into `nextToken`.

As the parser navigates through the grammatical rules, token advancement is strictly controlled by the `match()` helper function. When a parsing subroutine expects a specific structural terminal (such as a semicolon `;` or a closing parenthesis `)`), it passes the expected token code to `match()`. If `nextToken` matches the expected value, the subroutine consumes it by calling `lex()`, which automatically updates `nextToken` with the subsequent token from the source code stream. If a mismatch occurs, the execution immediately shifts to error handling.

2. Structural Routing and Statement Subroutines

The entry point of the structural analysis is the `program()` function, which acts as the root of the parsing tree and executes a continuous loop processing statements until the `EOF_CODE` terminal is encountered. The parser routes control using a dedicated layout within the `stmt()` subroutine:

- **declaration():** Triggered when `nextToken` matches `INT_TYPE` (`tam_sayi`). It ensures proper syntax for variable instantiation matching the `<id> "`;" configuration and registers the identifier into the symbol table.

- **assign():** Executed when an identifier (IDENT) is found at the start of a statement. It validates the presence of the assignment operator (ASSIGN_OP) and descends into the expression sub-tree before enforcing a trailing semicolon.
- **if_stmt() and optional():** Implements the conditional selection logic. The if_stmt() routine processes the eğer construct, parses the condition enclosed in parentheses, and executes the body statement. To resolve the dangling-else problem and support nested branching, the parser enters the recursive optional() subroutine. This routine evaluates whether the program continues into an ise (else-if) branch—recursively re-invoking optional()—or terminates at a değilse (else) block, cleanly accommodating an empty epsilon (ϵ) production when no further branches exist.
- **loop_stmt():** Manages iterative control flow. It dynamically branches into either a while-loop structure (iken) or a counter-controlled for-loop structure (için). The complex için framework sequentially calls begin_index() to parse initializations, evaluates the boundary loop condition(), executes the step operation(), and finally parses the loop body statement.

3. Expression Precedence and Factor Evaluation

Mathematical computations are parsed using hierarchical subroutines designed to inherently enforce operator precedence without requiring an abstract syntax tree transformation:

- **expr():** Handles lowest-precedence operations, specifically addition (+) and subtraction (-), by establishing a left-associative loop over sequential terms.
- **term():** Manages intermediate-precedence operations, namely multiplication (*) and division (/), isolating factors before returning control to the additive layer.
- **factor():** Represents the highest-precedence terminal layer. It evaluates whether an incoming structure is an identifier (IDENT), a numeric literal (INT_LIT), or a nested sub-expression enclosed in parentheses (<expr>), in which case it recursively restarts the expression evaluation chain.

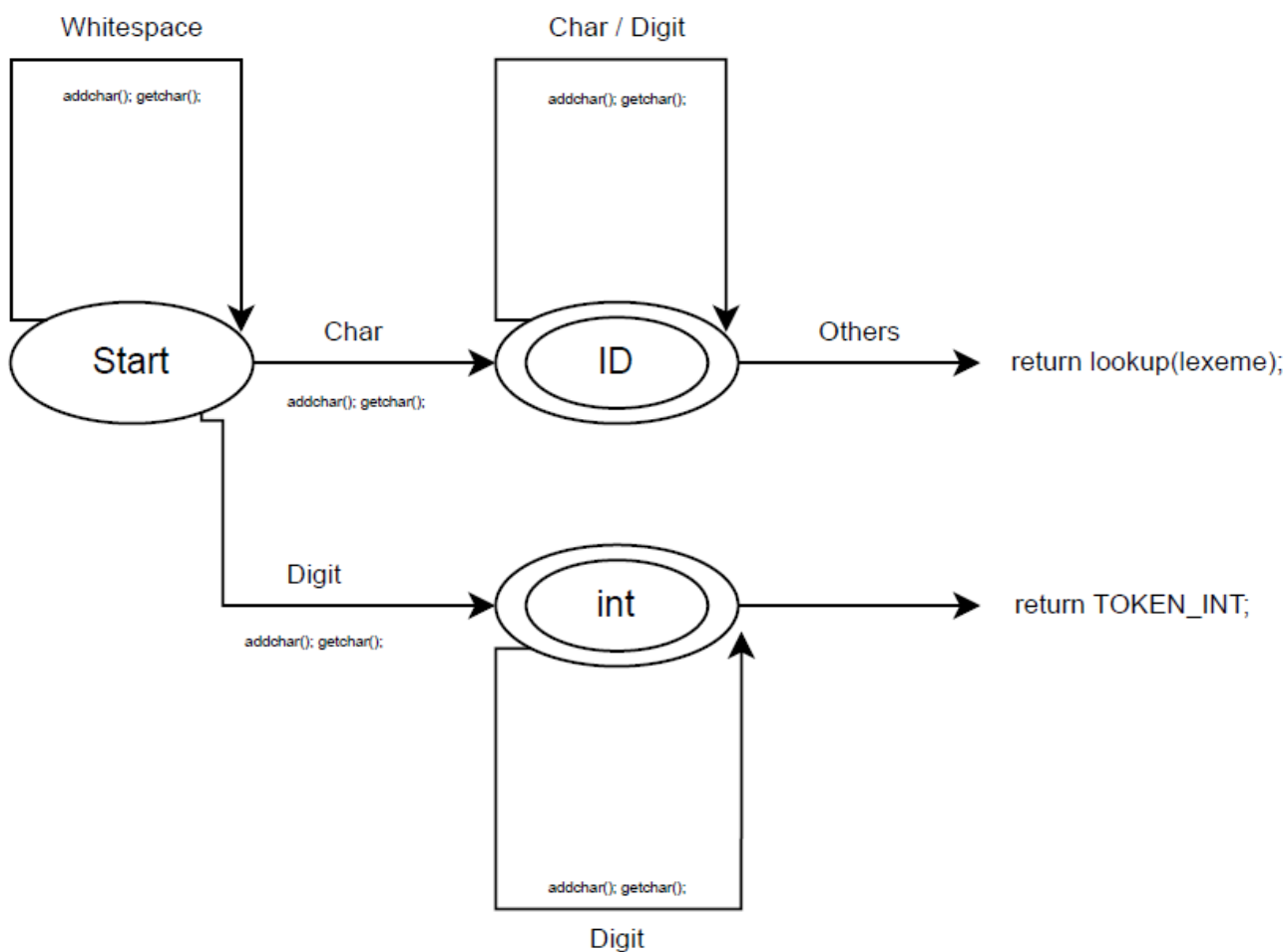
4. Syntax Error Interception and Diagnostics

When an incoming token deviates from the sequence prescribed by the BNF grammar rules, the parser drops out of its execution flow and calls the error() function. Rather than allowing the compiler frontend to crash or enter an infinite loop, the diagnostic utility prints a prominent compilation alert to the standard error stream (stderr), specifying the exact illegal lexeme that triggered the fault. It then terminates the program execution gracefully

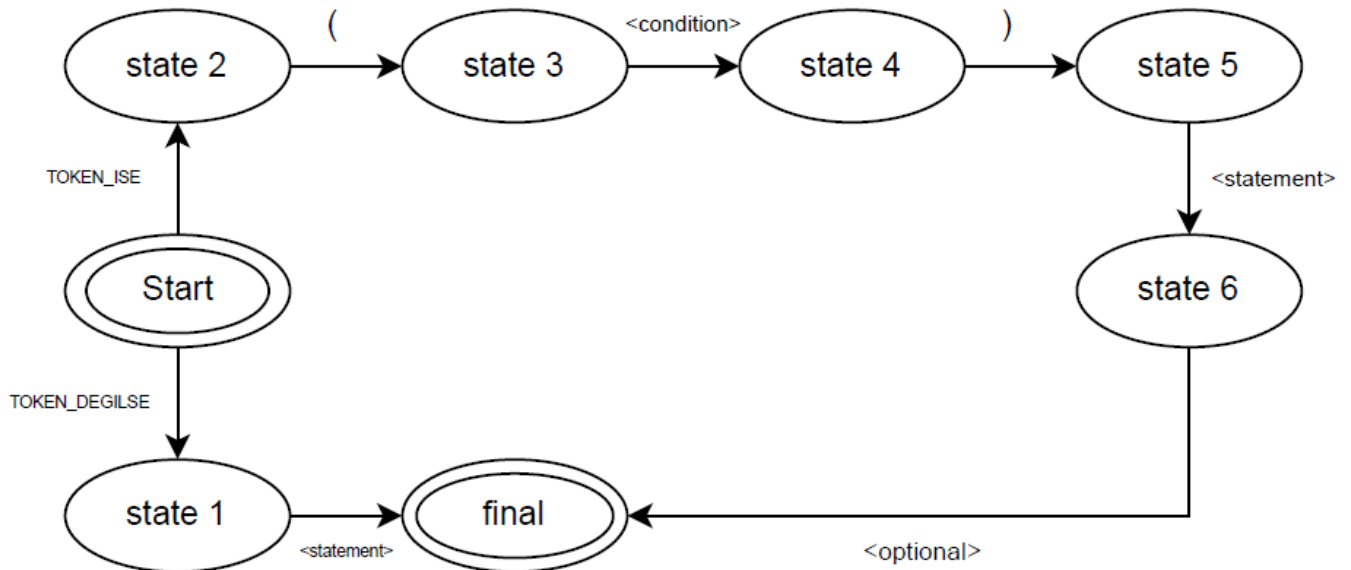
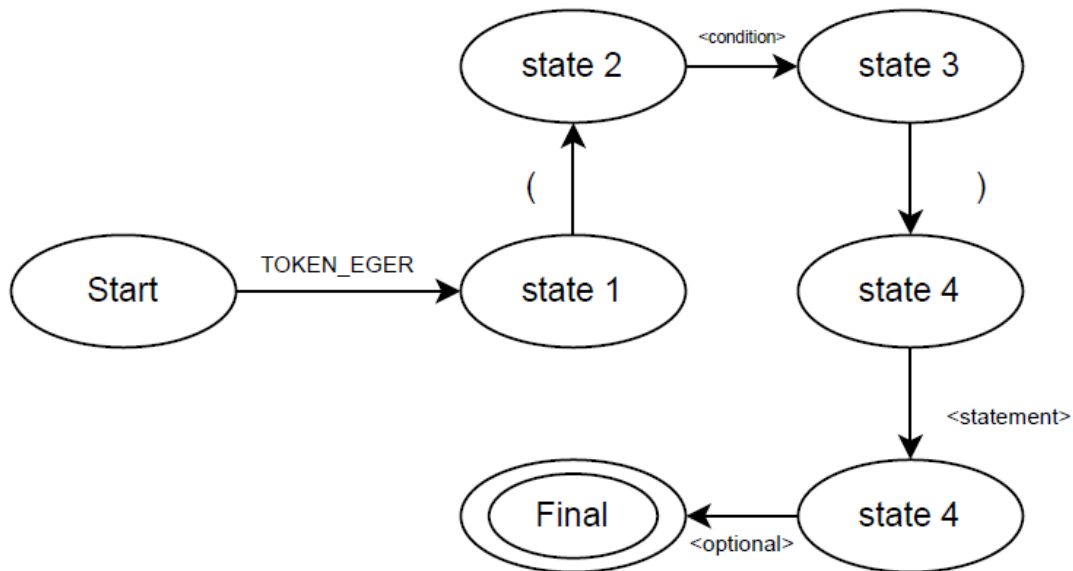
using `exit(1)`, providing immediate, actionable feedback regarding the location and nature of the syntactic violation.

State Diagrams

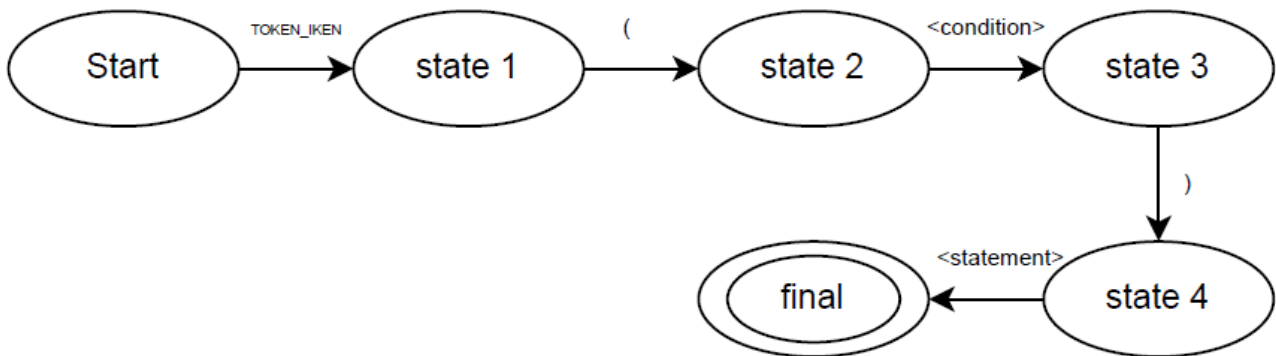
Identifier & Integer Literal Recognition:



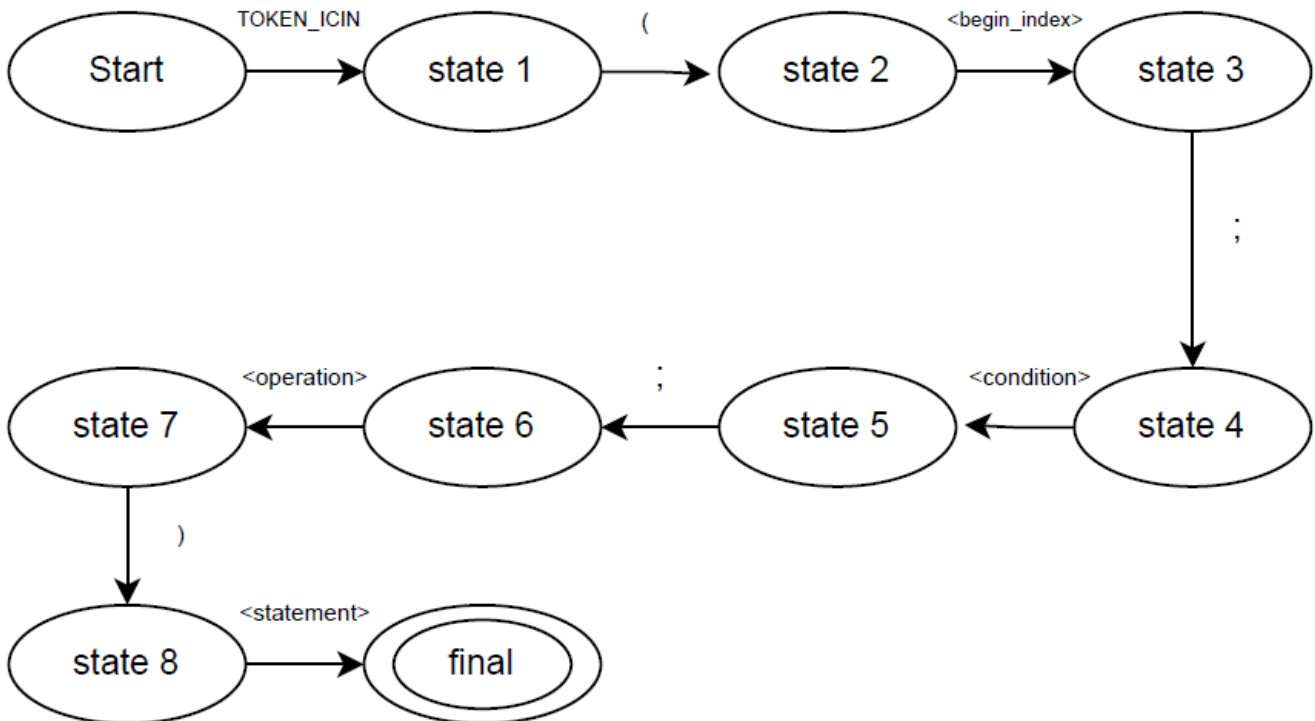
Conditional Branching Syntax Flow (eğer):



Loop Structure (iken):



Loop Structure (için):



Symbol Table / Lookup Table

The symbol table operates as a central repository for identifier management within the **C-TR-L** compiler frontend. While the primary role of the recursive-descent parser is syntactic validation, the symbol table lays the critical groundwork for static semantic analysis by securely tracking user-defined variables throughout the program's lifecycle.

1. Data Structure and Memory Allocation To maintain a lightweight and predictable execution footprint without relying on dynamic memory allocation, the symbol table is implemented as a statically allocated two-dimensional character array (`char symbolTable[MAX_SYMBOLS][100]`). This matrix architecture allows the compiler to store up to 100 distinct alphanumeric lexemes, each accommodating a maximum string length of 99 characters. A global state tracker (`symbolCount`) continuously monitors the active population of the table, shielding the system against index out-of-bounds errors and buffer overflows.

2. Insertion Mechanism and Collision Detection The population of the symbol table is strictly regulated by the `addSymbol()` subroutine. Before a newly encountered identifier is written into the repository, the function performs a linear sequential search across the existing entries using the `strcmp()` utility. If the identifier is already present in the matrix, the function safely returns without modifying the array, effectively preventing namespace collisions and duplicate memory allocations. If the identifier is distinct, it is safely committed to the matrix via `strcpy()`, the global counter is incremented, and a real-time diagnostic log is printed to the console.

3. Contextual Triggers and Disambiguation Strategy Unlike traditional scanners that might blindly push all strings into a lookup table, the **C-TR-L** frontend utilizes highly contextual hooks. The `addSymbol()` function is explicitly triggered by the syntax analyzer during specific grammatical events—namely within the `declaration()`, `assign()`, `begin_index()`, and `factor()` subroutines—whenever a valid IDENT token is successfully parsed.

Furthermore, strict token disambiguation is guaranteed at the lexical layer. Because reserved language commands (e.g., `tam_sayı`, `eğer`, `için`) are intercepted and uniquely tokenized by the lexer's `isKeyword()` filter before they ever reach the parser, it is structurally impossible for reserved keywords to pollute the symbol table. Upon the successful conclusion of syntax analysis, the compiler gracefully dumps the final, verified contents of the symbol table to the standard output stream for developer inspection.

Sample Source Programs and Execution Verification

To rigorously evaluate the structural integrity and diagnostic capabilities of the **C-TR-L** compiler frontend, a comprehensive testing suite comprising five distinct source code files was developed. Four of these files contain valid syntax designed to test specific grammatical modules, while the fifth file introduces a deliberate lexical anomaly to verify the parser's error-interception mechanisms.

Below are the source code files and their corresponding execution outputs generated by our compiler.

8.1 Test Case 1: Variable Declarations and Assignments

Objective: To verify that the frontend correctly parses primitive integer declarations (tam_sayı), initializes variables via the assignment operator (=), and successfully registers these identifiers into the Symbol Table.

Source Code (Variable_Declaration_and_Assignments.txt):

Plaintext

```
tam_sayı x;
```

```
tam_sayı y;
```

```
tam_sayı toplam;
```

```
x = 7;
```

```
y = 6;
```

```
toplam = x + y;
```

Execution Output:

```
C:\Users\Cengiz\Desktop\Programming Languages Project>parser Variable_Declaration_and_Assignments.txt
--- Lexical & Syntax Analysis Started ---
▶ Enter <program>
  ▶ Enter <stmt>
    ▶ Enter <declaration>
      [SYMBOL TABLE] Added identifier: x
    ✓ Exit <declaration>
  ✓ Exit <stmt>
  ▶ Enter <stmt>
    ▶ Enter <declaration>
      [SYMBOL TABLE] Added identifier: y
    ✓ Exit <declaration>
  ✓ Exit <stmt>
  ▶ Enter <stmt>
    ▶ Enter <declaration>
      [SYMBOL TABLE] Added identifier: toplam
    ✓ Exit <declaration>
  ✓ Exit <stmt>
  ▶ Enter <stmt>
    ▶ Enter <assign>
      ▶ Enter <expr>
        ▶ Enter <term>
          ▶ Enter <factor>
            ✓ Exit <factor>
          ✓ Exit <term>
        ✓ Exit <expr>
      ✓ Exit <assign>
    ✓ Exit <stmt>
  ▶ Enter <stmt>
    ▶ Enter <assign>
      ▶ Enter <expr>
        ▶ Enter <term>
          ▶ Enter <factor>
            ✓ Exit <factor>
          ✓ Exit <term>
        ✓ Exit <expr>
      ✓ Exit <assign>
    ✓ Exit <stmt>
  ▶ Enter <stmt>
    ▶ Enter <assign>
      ▶ Enter <expr>
        ▶ Enter <term>
          ▶ Enter <factor>
            ✓ Exit <factor>
          ✓ Exit <term>
        ✓ Exit <expr>
      ✓ Exit <assign>
    ✓ Exit <stmt>
```

```
    ✓ Exit <term>
  ▶ Enter <term>
    ▶ Enter <factor>
      ✓ Exit <factor>
    ✓ Exit <term>
  ✓ Exit <expr>
✓ Exit <assign>
✓ Exit <stmt>
✓ Exit <program> - Parsing successful!
--- Analysis Complete ---

--- Final Symbol Table ---
[0] x
[1] y
[2] toplam
```


8.2 Test Case 2: Arithmetic Expressions

Objective: To ensure that the recursive-descent parser implicitly respects mathematical operator precedence across basic binary operations (+, -, *, /) within assignment statements.

Source Code (Arithmetic_Expressions.txt):

Plaintext

```
tam_sayı x;
```

```
tam_sayı y;
```

```
tam_sayı sonuç;
```

```
x = 20;
```

```
y = 4;
```

```
sonuç = x + y;
```

```
sonuç = x - y;
```

```
sonuç = x * y;
```

```
sonuç = x / y;
```

Execution Output:

```
C:\Users\Cengiz\Desktop\Programming Languages Project>parser Arithmetic_Expressions.txt
--- Lexical & Syntax Analysis Started ---
▶ Enter <program>
  ▶ Enter <stmt>
    ▶ Enter <declaration>
      [SYMBOL TABLE] Added identifier: x
    ✓ Exit <declaration>
  ✓ Exit <stmt>
  ▶ Enter <stmt>
    ▶ Enter <declaration>
      [SYMBOL TABLE] Added identifier: y
    ✓ Exit <declaration>
  ✓ Exit <stmt>
  ▶ Enter <stmt>
    ▶ Enter <declaration>
      [SYMBOL TABLE] Added identifier: sonuç
    ✓ Exit <declaration>
  ✓ Exit <stmt>
  ▶ Enter <stmt>
    ▶ Enter <assign>
      ▶ Enter <expr>
        ▶ Enter <term>
          ▶ Enter <factor>
            ✓ Exit <factor>
          ✓ Exit <term>
        ✓ Exit <expr>
      ✓ Exit <assign>
    ✓ Exit <stmt>
  ▶ Enter <stmt>
    ▶ Enter <assign>
      ▶ Enter <expr>
        ▶ Enter <term>
          ▶ Enter <factor>
            ✓ Exit <factor>
          ✓ Exit <term>
        ✓ Exit <expr>
      ✓ Exit <assign>
    ✓ Exit <stmt>
  ▶ Enter <stmt>
    ▶ Enter <assign>
      ▶ Enter <expr>
        ▶ Enter <term>
          ▶ Enter <factor>
            ✓ Exit <factor>
          ✓ Exit <term>
        ▶ Enter <term>
          ▶ Enter <factor>
            ✓ Exit <factor>
          ✓ Exit <term>
        ✓ Exit <expr>
      ✓ Exit <assign>
    ✓ Exit <stmt>
```

```

✓ Exit <stmt>
▶ Enter <stmt>
  ▶ Enter <assign>
    ▶ Enter <expr>
      ▶ Enter <term>
        ▶ Enter <factor>
          ✓ Exit <factor>
        ✓ Exit <term>
      ▶ Enter <term>
        ▶ Enter <factor>
          ✓ Exit <factor>
        ✓ Exit <term>
      ✓ Exit <expr>
    ✓ Exit <assign>
  ✓ Exit <stmt>
▶ Enter <stmt>
  ▶ Enter <assign>
    ▶ Enter <expr>
      ▶ Enter <term>
        ▶ Enter <factor>
          ✓ Exit <factor>
        ▶ Enter <factor>
          ✓ Exit <factor>
        ✓ Exit <term>
      ✓ Exit <expr>
    ✓ Exit <assign>
  ✓ Exit <stmt>
▶ Enter <stmt>
  ▶ Enter <assign>
    ▶ Enter <expr>
      ▶ Enter <term>
        ▶ Enter <factor>
          ✓ Exit <factor>
        ▶ Enter <factor>
          ✓ Exit <factor>
        ✓ Exit <term>
      ✓ Exit <expr>
    ✓ Exit <assign>
  ✓ Exit <stmt>
✓ Exit <program> - Parsing successful!
--- Analysis Complete ---

--- Final Symbol Table ---
[0] x
[1] y
[2] sonuç

```

8.3 Test Case 3: Conditional Branching

Objective: To validate the resolution of the dangling-else ambiguity and test multi-way logical routing using the eğer, ise, and değilse constructs alongside relational boundary evaluations (<, ==).

Source Code (Conditional_Statements.txt):

Plaintext

```
tam_sayı x;
```

```
tam_sayı y;
```

```
x = 10;
```

```
y = 12;
```

```
eğer (x < y)
```

```
x = x+1;
```

```
ise ( x == y)
```

```
x = x+2;
```

```
değilse
```

```
x = x-1;
```

Execution Output:

```
C:\Users\Cengiz\Desktop\Programming Languages Project>parser Conditional_Statements.txt
--- Lexical & Syntax Analysis Started ---
▶ Enter <program>
  ▶ Enter <stmt>
    ▶ Enter <declaration>
      [SYMBOL TABLE] Added identifier: x
    ✓ Exit <declaration>
  ✓ Exit <stmt>
  ▶ Enter <stmt>
    ▶ Enter <declaration>
      [SYMBOL TABLE] Added identifier: y
    ✓ Exit <declaration>
  ✓ Exit <stmt>
  ▶ Enter <stmt>
    ▶ Enter <assign>
      ▶ Enter <expr>
        ▶ Enter <term>
          ▶ Enter <factor>
            ✓ Exit <factor>
          ✓ Exit <term>
        ✓ Exit <expr>
      ✓ Exit <assign>
    ✓ Exit <stmt>
  ▶ Enter <stmt>
    ▶ Enter <assign>
      ▶ Enter <expr>
        ▶ Enter <term>
          ▶ Enter <factor>
            ✓ Exit <factor>
          ✓ Exit <term>
        ✓ Exit <expr>
      ✓ Exit <assign>
    ✓ Exit <stmt>
  ▶ Enter <stmt>
    ▶ Enter <if_stmt>
      ▶ Enter <condition>
        ▶ Enter <expr>
          ▶ Enter <term>
            ▶ Enter <factor>
              ✓ Exit <factor>
            ✓ Exit <term>
          ✓ Exit <expr>
        ▶ Enter <rel_op>
          ✓ Exit <rel_op>
        ▶ Enter <expr>
          ▶ Enter <term>
            ▶ Enter <factor>
              ✓ Exit <factor>
            ✓ Exit <term>
          ✓ Exit <expr>
        ✓ Exit <condition>
```

```

▶ Enter <stmt>
  ▶ Enter <assign>
    ▶ Enter <expr>
      ▶ Enter <term>
        ▶ Enter <factor>
          ✓ Exit <factor>
        ✓ Exit <term>
      ▶ Enter <term>
        ▶ Enter <factor>
          ✓ Exit <factor>
        ✓ Exit <term>
      ✓ Exit <expr>
    ✓ Exit <assign>
  ✓ Exit <stmt>
  ▶ Enter <optional>
    ▶ Enter <condition>
      ▶ Enter <expr>
        ▶ Enter <term>
          ▶ Enter <factor>
            ✓ Exit <factor>
          ✓ Exit <term>
        ✓ Exit <expr>
      ▶ Enter <rel_op>
        ✓ Exit <rel_op>
      ▶ Enter <expr>
        ▶ Enter <term>
          ▶ Enter <factor>
            ✓ Exit <factor>
          ✓ Exit <term>
        ✓ Exit <expr>
      ✓ Exit <condition>
    ▶ Enter <stmt>
      ▶ Enter <assign>
        ▶ Enter <expr>
          ▶ Enter <term>
            ▶ Enter <factor>
              ✓ Exit <factor>
            ✓ Exit <term>
          ▶ Enter <term>
            ▶ Enter <factor>
              ✓ Exit <factor>
            ✓ Exit <term>
          ✓ Exit <expr>
        ✓ Exit <assign>
      ✓ Exit <stmt>
    ✓ Exit <optional>
  ▶ Enter <stmt>
    ▶ Enter <assign>
      ▶ Enter <expr>
        ▶ Enter <term>
          ▶ Enter <factor>

```

```

    ✓ Exit <factor>
    ✓ Exit <term>
    ▶ Enter <term>
    ▶ Enter <factor>
    ✓ Exit <factor>
    ✓ Exit <term>
    ✓ Exit <expr>
    ✓ Exit <assign>
    ✓ Exit <stmt>
    ✓ Exit <optional>
    ✓ Exit <optional>
    ✓ Exit <if_stmt>
    ✓ Exit <stmt>
✓ Exit <program> - Parsing successful!
--- Analysis Complete ---

--- Final Symbol Table ---
[0] x
[1] y

```

8.4 Test Case 4: Iterative Loop Structures

Objective: To verify the syntactic parsing of complex block structures, specifically pre-tested conditional loops (iken) and counter-controlled initialization loops (için) with unary increment operations (++).

Source Code (Loop_Structures.txt):

Plaintext

tam_sayı i;

tam_sayı döngü_toplamı;

i = 0;

döngü_toplamı = 0;

iken(i < 5)

i = i + 1;

için (i = 0; i < 10; i++)

döngü_toplamı = döngü_toplamı + i;

Execution Output:

```
C:\Users\Cengiz\Desktop\Programming Languages Project>parser Loop_Structures.txt
--- Lexical & Syntax Analysis Started ---
▶ Enter <program>
  ▶ Enter <stmt>
    ▶ Enter <declaration>
      [SYMBOL TABLE] Added identifier: i
    ✓ Exit <declaration>
  ✓ Exit <stmt>
  ▶ Enter <stmt>
    ▶ Enter <declaration>
      [SYMBOL TABLE] Added identifier: döngü_toplamı
    ✓ Exit <declaration>
  ✓ Exit <stmt>
  ▶ Enter <stmt>
    ▶ Enter <assign>
      ▶ Enter <expr>
        ▶ Enter <term>
          ▶ Enter <factor>
            ✓ Exit <factor>
          ✓ Exit <term>
        ✓ Exit <expr>
      ✓ Exit <assign>
    ✓ Exit <stmt>
  ▶ Enter <stmt>
    ▶ Enter <assign>
      ▶ Enter <expr>
        ▶ Enter <term>
          ▶ Enter <factor>
            ✓ Exit <factor>
          ✓ Exit <term>
        ✓ Exit <expr>
      ✓ Exit <assign>
    ✓ Exit <stmt>
  ▶ Enter <stmt>
    ▶ Enter <loop_stmt>
      ▶ Enter <condition>
        ▶ Enter <expr>
          ▶ Enter <term>
            ▶ Enter <factor>
              ✓ Exit <factor>
            ✓ Exit <term>
          ✓ Exit <expr>
        ▶ Enter <rel_op>
          ✓ Exit <rel_op>
        ▶ Enter <expr>
          ▶ Enter <term>
            ▶ Enter <factor>
              ✓ Exit <factor>
            ✓ Exit <term>
          ✓ Exit <expr>
      ✓ Exit <loop_stmt>
    ✓ Exit <stmt>
```



```

    ✓ Exit <condition>
▶ Enter <stmt>
  ▶ Enter <assign>
    ▶ Enter <expr>
      ▶ Enter <term>
        ▶ Enter <factor>
          ✓ Exit <factor>
        ✓ Exit <term>
      ▶ Enter <term>
        ▶ Enter <factor>
          ✓ Exit <factor>
        ✓ Exit <term>
      ✓ Exit <expr>
    ✓ Exit <assign>
  ✓ Exit <stmt>
  ✓ Exit <loop_stmt>
  ✓ Exit <stmt>
▶ Enter <stmt>
  ▶ Enter <loop_stmt>
    ▶ Enter <begin_index>
      ▶ Enter <expr>
        ▶ Enter <term>
          ▶ Enter <factor>
            ✓ Exit <factor>
          ✓ Exit <term>
        ✓ Exit <expr>
      ✓ Exit <begin_index>
    ▶ Enter <condition>
      ▶ Enter <expr>
        ▶ Enter <term>
          ▶ Enter <factor>
            ✓ Exit <factor>
          ✓ Exit <term>
        ✓ Exit <expr>
      ▶ Enter <rel_op>
        ✓ Exit <rel_op>
      ▶ Enter <expr>
        ▶ Enter <term>
          ▶ Enter <factor>
            ✓ Exit <factor>
          ✓ Exit <term>
        ✓ Exit <expr>
      ✓ Exit <condition>
    ▶ Enter <operation>
      ✓ Exit <operation>
  ▶ Enter <stmt>
    ▶ Enter <assign>
      ▶ Enter <expr>
        ▶ Enter <term>
          ▶ Enter <factor>
            ✓ Exit <factor>

```

```

    ✓ Exit <term>
    ▶ Enter <term>
      ▶ Enter <factor>
        ✓ Exit <factor>
      ✓ Exit <term>
    ✓ Exit <expr>
  ✓ Exit <assign>
✓ Exit <stmt>
  ✓ Exit <loop_stmt>
  ✓ Exit <stmt>
✓ Exit <program> - Parsing successful!
--- Analysis Complete ---

--- Final Symbol Table ---
[0] i
[1] döngü_toplamı

```

8.5 Test Case 5: Syntax Error Interception (Invalid File)

Objective: To prove that the lexer and parser safely halt execution upon encountering an illegal/foreign character, rather than crashing or entering an infinite loop. The system must produce a meaningful diagnostic error message.

Source Code (Invalid_File.txt):

Plaintext

tam_sayı x;

x = 3;

eğer (x < 4)

x = x € 2;

Execution Output:

```
C:\Users\Cengiz\Desktop\Programming Languages Project>parser Invalid_File.txt
--- Lexical & Syntax Analysis Started ---
▶ Enter <program>
  ▶ Enter <stmt>
    ▶ Enter <declaration>
      [SYMBOL TABLE] Added identifier: x
    ✓ Exit <declaration>
  ✓ Exit <stmt>
  ▶ Enter <stmt>
    ▶ Enter <assign>
      ▶ Enter <expr>
        ▶ Enter <term>
          ▶ Enter <factor>
            ✓ Exit <factor>
          ✓ Exit <term>
        ✓ Exit <expr>
      ✓ Exit <assign>
    ✓ Exit <stmt>
  ▶ Enter <stmt>
    ▶ Enter <if_stmt>
      ▶ Enter <condition>
        ▶ Enter <expr>
          ▶ Enter <term>
            ▶ Enter <factor>
              ✓ Exit <factor>
            ✓ Exit <term>
          ✓ Exit <expr>
        ▶ Enter <rel_op>
          ✓ Exit <rel_op>
        ▶ Enter <expr>
          ▶ Enter <term>
            ▶ Enter <factor>
              ✓ Exit <factor>
            ✓ Exit <term>
          ✓ Exit <expr>
        ✓ Exit <condition>
      ▶ Enter <stmt>
        ▶ Enter <assign>
          ▶ Enter <expr>
            ▶ Enter <term>
              ▶ Enter <factor>
                ✓ Exit <factor>
              ✓ Exit <term>
            ✓ Exit <expr>
          ✓ Exit <assign>
        ✓ Exit <stmt>
      ✓ Exit <if_stmt>
    ✓ Exit <stmt>
  ✓ Exit <program>
✗ SYNTAX ERROR at '€'
  Parser expected a valid token according to the grammar.
```

Aptitude Verdict: As demonstrated in the output above, the structural analyzer successfully terminated parsing execution exactly at the illegal currency literal zone, generating a precise SYNTAX ERROR at '€' exception message and preventing further compiler pipeline corruption.

Conclusion

The implementation of the **C-TR-L** compiler pipeline successfully demonstrates the practical execution of formal translation paradigms outlined in language theory. By synthesizing a state-driven lexical analyzer and a top-down recursive-descent parser, the project translates theoretical principles of Context-Free Grammars (CFGs) and regular expressions into a functional, diagnostic software tool.

Throughout the development lifecycle, several critical software engineering challenges were addressed. The primary lexical milestone was the resolution of multi-byte UTF-8 character mapping via a custom classification routine, enabling seamless parsing of localized Turkish graphemes without compromising the state transitions of the deterministic finite automaton. On the syntactic layer, the parser effectively enforced mathematical operator precedence directly through layered, left-associative functions (`expr`, `term`, `factor`) and resolved the dangling-else structural ambiguity in nested conditional blocks via recursive routing inside the `optional()` subroutine.

Additionally, the real-time collision checking and management within the centralized static Symbol Table confirmed the implementation's capability to track state boundaries and lay a stable foundation for static semantic verification.

While the current architecture delivers a robust frontend framework restricted to primitive integer typologies (`tam_sayı`) and elementary imperative control structures, it serves as an extensible blueprint for a full-scale compiler system. Future engineering iterations could significantly expand the language's scope by introducing a dynamic semantic analyzer to enforce compile-time type-safety rules, broadening the type matrix to support floating-point and string literals, and appending a backend target-code generation utility to translate the verified syntactic structures into executable machine code or assembly languages. Ultimately, **C-TR-L** proves that integrating localized linguistic tokens into an imperative programming design reduces cognitive overhead for native speakers while strictly maintaining architectural and mathematical rigor.